08-CTO-NEXT-STEPS.md

.things that Matter

CTO Next Steps & Action Plan

Document Information	
Company Name	.things that Matter
Document Type	CTO Action Plan
Version	1.0
Date	January 21, 2026
Author	Matome Mboweni, CTO

Executive Summary

\1 Sprint 0 (Setup Phase) - Week 1, Day 1
\1 Phase 0 Go/No-Go Decision (Feb 28, 2026)
\1 11 months to public launch (Dec 15, 2026)

This document outlines immediate next steps, priorities, and action items for the CTO to execute over the next 2-4 weeks.

Immediate Priorities (This Week: Jan 21-27, 2026)

Priority 1: Complete Sprint 0 Setup Tasks

Task	Status	Deadline	Owner	Notes
Sign new company contract	Pending	Jan 31, 2026	CEO + CTO	Legal foundation
Set up development environment	In Progress	Feb 2, 2026	CTO	Cursor Pro, VS Code, Rust
Create GitHub organization	Pending	Jan 25, 2026	CTO	`thingsthatmatter` org
Initialize monorepo structure	Pending	Jan 26, 2026	CTO	`packages/` directory
Set up Supabase project	Pending	Jan 28, 2026	CTO	Database schema
Configure Vercel deployment	Pending	Jan 29, 2026	CTO	CI/CD pipeline
Set up Cloudflare R2 bucket	Pending	Jan 30, 2026	CTO	Artifact storage
Begin Rust learning	In Progress	Ongoing	CTO	Chapters 1-5

Priority 2: Infrastructure Accounts Setup

\1

1. \1 (if not already done):
- GitHub Organization: \1
- Supabase account (free tier)
- Vercel account (hobby tier)
- Cloudflare account (R2 storage)
- Replicate account (CLIP API)
- Redis Cloud (free tier)

2. \1 for all paid services:
- Cloudflare R2: ~R630/month limit
- Replicate CLIP: ~R540/month limit

3. \1
- Store securely in environment variables
- Document in secure password manager

Week 2 Priorities (Jan 28 - Feb 2, 2026)

Sprint 0 Completion Checklist

- [ ] All infrastructure accounts created and verified
- [ ] Development environment fully operational
- [ ] GitHub repo structure established
- [ ] Database schema designed (initial draft)
- [ ] CI/CD pipeline configured
- [ ] Rust toolchain installed (\1, \1)
- [ ] Rust Book: Completed Chapters 1-5

Phase 0 Preparation

\1

1. \1
- Research XMP metadata injection libraries (Rust crates)
- Research EOF binary append techniques
- Identify test platforms (Discord, Telegram, Twitter, etc.)
- Create Phase 0 testing plan

2. \1
- Set up Rust project for steganography tools
- Identify crates: \1 (image processing), \1 (XMP), etc.
- Create test image samples

Phase 0: Critical Validation (Feb 3 - Mar 2, 2026)

Week 3-4: Build Steganography Tools

\1
- [ ] \1 - XMP metadata injection tool
- [ ] \1 - EOF binary append tool
- [ ] \1 - Payload extraction tool
- [ ] Initial platform tests (5 platforms)

\1

```
// Example structure
xmp_inject.rs:
```

```
  - Read image (JPEG/PNG) using `image` crate
  - Inject XMP metadata with hash + gist
  - Save as polyglot image

eof_append.rs:
  - Read image
  - Append .thing binary to EOF
  - Validate file still displays correctly

extract_payload.rs:
  - Extract XMP beacon from image header
  - Extract EOF binary payload
  - Validate both are intact
```

Week 5: End-to-End Prototype

\1

```
Product JSON → Compiler → .thing File → Polyglot Image → Platform Upload → Extract → Verify
```

\1
1. Basic product JSON validator (using \1 + \1)
2. Simple FlatBuffers serializer (Rust)
3. Ed25519 signing (using \1 crate)
4. CLIP API integration test (using \1)
5. Image steganography pipeline (using \1 crate)
6. Extraction and verification

\1
- End-to-end flow works locally
- Can extract and verify payload after platform upload
- Document any issues or limitations

Week 6: Platform Testing & Go/No-Go Decision

\1

```
| Platform | Test Method | Expected Result | Actual Result | Survival? |
|----------|-------------|-----------------|---------------|-----------|
| Discord | Upload/download | XMP + EOF intact | TBD |  |
| Telegram | Send/receive | XMP + EOF intact | TBD |  |
| Twitter/X | Tweet with image | XMP may strip | TBD |  |
| Instagram | Post image | Likely strip | TBD |  |
| ... | ... | ... | ... | ... |
```

\1

```
| Survival Rate | Decision | Action Plan |
|---------------|----------|-------------|
| ≥70% |  **GO** | Full steganography approach |
| 50-69% |  **HYBRID** | Steg + matter:// URIs |
| <50% |  **PIVOT** | matter:// as primary, steg secondary |
```

\1
- Testing results for all 20 platforms
- Survival rate calculation
- Cost analysis (CLIP API)
- Go/No-Go recommendation

- Updated roadmap (if pivot needed)

Technical Learning Path

Rust Learning Schedule

\1
- [ ] Complete Rust Book Chapters 1-5
- [ ] Set up Rust development environment
- [ ] Build simple CLI tool (hello world → file I/O)

\1
- [ ] Rust Book Chapters 6-10 (Ownership, Structs, Enums)
- [ ] Practice: Build simple FlatBuffers serializer
- [ ] Study \1 crate examples

\1
- [ ] Rust Book Chapters 11-15 (Testing, CLI, Collections)
- [ ] Study \1 for WASM compilation
- [ ] Review existing Rust commerce/blockchain projects

\1
- Practice Rust daily (even if just 30 minutes)
- Join Rust Discord/forums for questions
- Read Rust blogs and watch tutorials

Rust Implementation Focus (Sprint 2-3)

\1 Build Rust compiler from the start, leveraging C/C++ background

\1
- \1 + \1 for validation
- \1 (Rust crate)
- \1 for Ed25519 signing
- \1 crate for image manipulation
- \1 or \1 for XMP metadata
- \1 for compression
- \1 for CLIP API calls
- \1 for async operations

Infrastructure Setup Details

1. Supabase Project Setup

\1
1. Create new Supabase project
2. Design initial schema:

```
-- Merchants table
```

```
-- Products table
-- ThingFiles table (hash → metadata)
-- Resolution logs
```

3. Set up Row Level Security (RLS)
4. Create API keys
5. Test connection from local dev

\1 See Technical Architecture doc (Section 2.6)

2. Vercel Project Setup

\1
1. Create Vercel account (connect GitHub)
2. Create new project
3. Set up environment variables:
- \1
- \1
- \1
- \1
- \1
- \1 (Ed25519)
4. Configure Edge Functions
5. Test deployment

3. Cloudflare R2 Setup

\1
1. Create Cloudflare account
2. Enable R2 (pay-as-you-go)
3. Create bucket: \1
4. Set up CORS for web access
5. Create public CDN endpoint
6. Test upload/download

\1 ~R630/month for 10K products (estimate)

4. Replicate API Setup

\1
1. Create Replicate account
2. Get API key
3. Test CLIP model:

```
use reqwest;
// Example API call structure
let response = reqwest::Client::new()
    .post("https://api.replicate.com/v1/predictions")
    .json(&json!({
        "version": "clip-vit-base-patch32",
        "input": {"image": image_url}
    }))
    .send()
    .await?;
```

4. Document cost per inference (~R0.40/image)

Risk Mitigation Actions

Immediate Risk Mitigations:

1. \1
- Document created (you're reading it)
- Batch work requests when possible
- Use slower requests for non-critical tasks
- Plan ahead to minimize back-and-forth

2. \1
- Start early (ongoing in Sprint 0)
- C/C++ background provides strong foundation
- Learn Rust as you build (practical approach)
- Allocate 3 hours/week minimum for learning

3. \1
- Set up billing alerts
- Monitor monthly spend closely
- Use free tiers wherever possible
- Keep 27% operating reserve untouched

4. \1
- Test early and often
- Have matter:// URI fallback ready
- Document all platform behaviors

Weekly Routine (Recommended)

Monday Morning (2 hours)
- Review previous week's progress
- Plan week ahead
- Update CEO on status
- Prioritize tasks

Tuesday-Thursday (20 hours)
- Core development work
- Infrastructure setup
- Learning (Rust, new tech)

Friday (3 hours)
- Code review & cleanup
- Documentation updates
- Progress tracking
- Next week prep

Weekend (Optional)
- Light reading/research

- Experimentation
- Rest (important!)

\1 ~30 hours/week (matches roadmap assumptions)

Communication Cadence

With CEO:

- \1 Every Monday (30 min)
- \1 First Monday of month
- \1 Immediate notification

Documentation:

- \1 Weekly (Friday)
- \1 Update roadmap status
- \1 Document major technical decisions

Success Metrics (Sprint 0)

```
| Metric | Target | Current | Status |
|--------|--------|---------|--------|
| Infrastructure accounts | 6/6 | ? |  |
| Development environment | 100% | ? |  |
| GitHub org created |  | ? |  |
| Rust Chapters completed | 5/5 | ? |  |
| Monorepo structure |  | ? |  |
| Supabase project |  | ? |  |
| Vercel deployment |  | ? |  |
| R2 bucket |  | ? |  |
```

Immediate Action Items (Today/Tomorrow)

Today (Jan 21, 2026):

- [x] Review this document
- [ ] Confirm contract signing timeline with CEO
- [ ] Create GitHub organization
- [ ] Set up Supabase account
- [ ] Install Rust toolchain (\1)
- [ ] Start Rust Book Chapter 1

Tomorrow (Jan 22, 2026):

- [ ] Set up Vercel account
- [ ] Create Cloudflare R2 bucket
- [ ] Design database schema (initial draft)
- [ ] Initialize monorepo structure
- [ ] Complete Rust Book Chapter 1-2

- [ ] Document any blockers

Decision Points Coming Up

Week 2 (Jan 28 - Feb 2):

- \1 (needed before Phase 0)
- \1 for implementation (research dependencies)
- \1 (Shadow Index API design)

Phase 0 Week 4 (Feb 24-28):

- \1 on steganography approach
- \1 if survival rate <50%
- \1 if approach changes

Resources & References

Documentation:

- \1 - Vision and objectives
- \1 - System design
- \1 - Requirements
- \1 - Detailed timeline
- \1 - Risk mitigation
- \1 - UML diagrams

External Resources:

- \1 (\2)
- \1 (\2)
- \1 (\2)
- \1 (\2)

Notes & Open Questions

Technical Questions to Resolve:

- [ ] Best Rust crate for XMP injection? (\1, \1, or custom?)
- [ ] FlatBuffers Rust crate performance acceptable?
- [ ] WASM size constraints for browser extension?
- [ ] Best steganography technique for JPEG vs PNG?
- [ ] Async runtime choice (\1 vs \1)?

Business Questions to Clarify with CEO:

- [ ] Contract signing timeline?
- [ ] Any changes to budget allocation?
- [ ] Priority adjustments based on market feedback?

- [ ] Partnership opportunities to explore?

Appendix: Quick Reference

Command Cheat Sheet:

```
# Rust
rustup install stable
cargo new matter-core --lib
cargo test

# Git
git init
git remote add origin https://github.com/thingsthatmatter/...
git branch -M main

# Rust
cargo new matter-core --lib
cargo add serde serde_json
cargo add flatbuffers
cargo add ed25519-dalek
cargo add image
cargo add reqwest --features json
cargo add tokio --features full
```

Environment Variables Template:

```
# .env file (never commit!)
SUPABASE_URL=https://xxx.supabase.co
SUPABASE_ANON_KEY=xxx
SUPABASE_SERVICE_KEY=xxx
CLOUDFLARE_R2_ACCOUNT_ID=xxx
CLOUDFLARE_R2_ACCESS_KEY_ID=xxx
CLOUDFLARE_R2_SECRET_ACCESS_KEY=xxx
REPLICATE_API_KEY=xxx
SIGNING_PRIVATE_KEY=xxx  # Ed25519 private key (base64)
```

\1 1.0
\1 January 21, 2026
\1 January 28, 2026 (End of Sprint 0, Week 1)

\1 You're building something revolutionary. Take it one step at a time, validate early, and don't be afraid to pivot if needed. The Phase 0 validation is critical—it will determine the entire technical approach.

\1